

快速傅里叶变换：定义和速度

快速傅里叶变换（Fast Fourier Transform, FFT）是一类快速计算序列的离散傅里叶变换（DFT）或其逆变换的方法。傅里叶分析把信号从原始域（通常是时间或空间）转换为频域表示，也可以进行反向转换。FFT 将 DFT 矩阵分解为稀疏因子的乘积，从而加快计算：对于大小为 n 的数据，直接按 DFT 定义计算的复杂度为 $O(n^2)$ ，使用 FFT 则可降至 $O(n \log n)$ 。

FFT 与直接按定义计算 DFT 得到的结果相同，主要区别在于计算速度。实际计算会受到舍入误差的影响，因此，许多 FFT 算法还可能比直接按定义求值更精确。

DFT 的复数输出记为 $X_0, \dots, X_{N-1}$ ，其中每个 X_k 按下式计算：

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}, \quad k = 0, \dots, N-1.$$

这个公式需要计算 N 个输出，每个输出都包含 N 项求和。如果直接逐项计算，每个输出需要 N 次复数乘法和 $N-1$ 次复数加法；全部 N 个输出合计需要 N^2 次复数乘法和 N^2-N 次复数加法，后者也可写成 $N(N-1)$ 。因此，直接计算 DFT 的运算复杂度为 $O(N^2)$ 。省去乘以 1 等简单运算，可以减少 $O(N)$ 次运算。

FFT 泛指能在 $O(N \log N)$ 次运算内算出相同结果的方法。例如，常见的基 2 Cooley-Tukey 算法在 N 为 2 的幂时，只需要 $(N/2) \log_2 N$ 次复数乘法和 $N \log_2 N$ 次复数加法。所有已知 FFT 算法的运算复杂度都是 $O(N \log N)$ ，但目前还没有证明表明更低的复杂度不可能实现。现代计算机的实际性能通常不只取决于算术运算速度，不过，从 $O(N^2)$ 降至 $O(N \log N)$ 仍然是很重要的改进。

FFT 广泛应用于工程、科学和数学领域。它的基本思想早在 1805 年就被推导出来，到 1965 年得到普及。1994 年，美国数学家 Gilbert Strang 将 FFT 称为“一生中最重要的数值算法”之一；IEEE 科学与工程计算期刊也将它列入了 20 世纪十大算法。